

JDBC 案例

今日内容介绍

- ◆ 案例：查询所有商品
- ◆ 案例：添加商品
- ◆ 案例：修改商品
- ◆ 案例：删除指定商品

今日内容学习目标

- ### ◆ 巩固 WEB 知识

第1章 案例：查询所有商品

1.1 案例介绍

后台管理程序中，添加商品已经完成，接着我们来完成查询所有。

查询条件

分类

商品名称:

查询

重置

商品列表

添加

缩略图	商品分类	商品名称	商品价格	描述	编辑	查看	删除
	服装	2016春秋鞋跟真皮工作鞋 中年单鞋防水台中跟女士皮鞋一字扣女神鞋	998	女神爆款			
信息展示							
	服装	2016春秋鞋跟真皮工作鞋 中年单鞋防水台中跟女士皮鞋一字扣女神鞋	990	女神爆款			

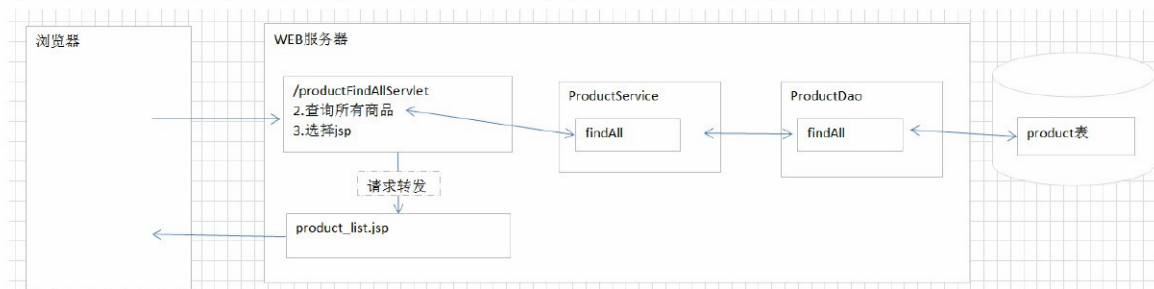
10

当前第1页, 共10页

当前显示: 到10条, 共114条记录

1.2 案例分析

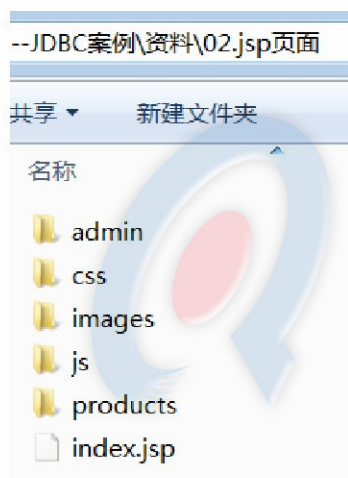
之前我们已经完成前台商品的查询，后台功能类似，只是展示页面不同。



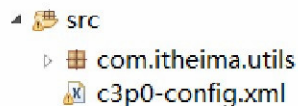
1.3 案例实现

1.3.1 搭建环境

- 拷贝 HTML 内容



- 工具类和 c3p0 配置文件



- 导入 jar 包



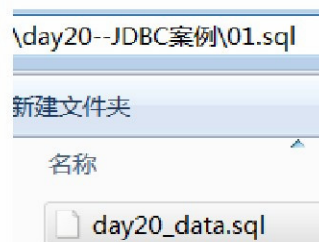
1.3.2 数据库及表

```
-- 创建数据库
drop database if exists `day20_db`;
create database `day20_db`;

-- 使用数据库
use day20_db;

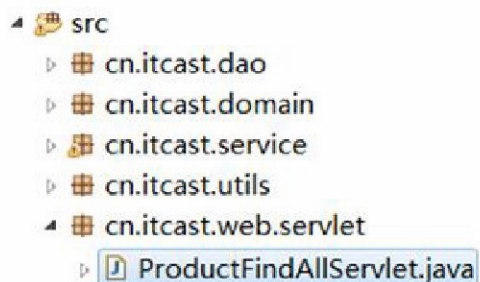
-- 创建商品表
CREATE TABLE `product` (
  `pid` varchar(32) NOT NULL,
  `pname` varchar(50) DEFAULT NULL,      #商品名称
  `market_price` double DEFAULT NULL,    #商场价
  `shop_price` double DEFAULT NULL,      #商城价
  `pimage` varchar(200) DEFAULT NULL,    #商品图片路径
  `pdate` date DEFAULT NULL,             #上架时间
  `is_hot` int(11) DEFAULT NULL,          #是否热门: 0=不热门,1=热门
  `pdesc` varchar(255) DEFAULT NULL,     #商品描述
  `pflag` int(11) DEFAULT 0,              #商品标记: 0=未下架(默认值),1=已经下架
  `cid` varchar(32) DEFAULT NULL,        #分类 id
  PRIMARY KEY (`pid`)
) ;
```

- 完整 SQL，参考《day20_data.sql》



1.3.5 编写 servlet

- 编写 servlet 实现类



```
public class ProductFindAllServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //1

        //2 通知 service, 查询所有商品
        ProductService productService = new ProductService();
        List<Product> allProduct = productService.findAll();

        //3 选择 jsp
        //3.1 将查询结果存放在 request 作用域
        request.setAttribute("allProduct", allProduct);
        //3.2 请求转发

        request.getRequestDispatcher("/admin/product/product_list.jsp").forward(request,
response);

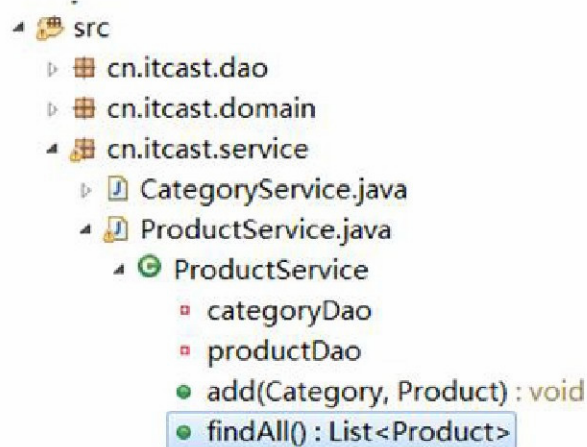
    }
}
```

- web.xml 配置

```
<servlet>
    <servlet-name>ProductFindAllServlet</servlet-name>
    <servlet-class>cn.itcast.web.servlet.ProductFindAllServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ProductFindAllServlet</servlet-name>
    <url-pattern>/productFindAllServlet</url-pattern>
</servlet-mapping>
```

1.3.6 service 层

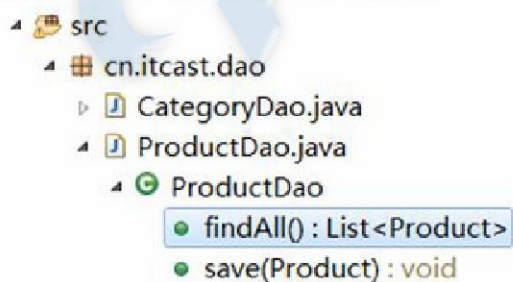
- 编写 ProductService，添加 findAll()方法



```
/**
 * 查询所有
 * @return
 */
public List<Product> findAll(){
    return productDao.findAll();
}
```

1.3.7 dao 层

- 编写 ProductDao，添加 findAll()方法



```
/**
 * 查询所有
 * @return
 */
public List<Product> findAll(){
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        String sql = "select * from product";
        Object[] params = {};
```

```
        return queryRunner.query(sql, new BeanListHandler<Product>(Product.class),
params);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

1.3.8 修改 JSP，显示数据

- jsp 页面数据显示

页面位置：/day13_findall/WebContent/admin/product/product_list.jsp

```
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!--列表项 start -->
<c:forEach items="${allProduct}" var="product">
    <tr onmouseover="this.style.backgroundColor = '#F5FAFE'"
        onmouseout="this.style.backgroundColor = '#fff';">
        <td style="CURSOR: hand; HEIGHT: 22px" align="center">
            
        </td>
        <td style="CURSOR: hand; HEIGHT: 22px" align="center">
            ${product.category_id}
        </td>
        <td style="CURSOR: hand; HEIGHT: 22px" align="center">
            ${product.pname}
        </td>
        <td style="CURSOR: hand; HEIGHT: 22px" align="center">
            ${product.price}
        </td>
        <td style="CURSOR: hand; HEIGHT: 22px" align="center">
            ${product.description}
        </td>
        <td align="center" style="HEIGHT: 22px">
            <a href="../user/edit.html?userID=15">
                
            </a>
        </td>
        <td align="center" style="HEIGHT: 22px">
            <a href="../user/view.html?userID=15">
                
    </a>
</td>
<td align="center" style="HEIGHT: 22px">
    <a href="../user/list.html?userID=15">
        
    </a>
</td>
</tr>
</c:forEach>
<%--列表项 end --%>
```

第2章 案例：添加商品

2.1 案例介绍

日常生活中，我们浏览电商（京东、淘宝等）网站时，浏览器的商品会被后台管理员提前录入到系统中，接下来，我们一起学习后台管理员添加商品的整个流程。添加的过程需要注意进行分类的选择，如果没有需要分类，可以同时添加。

添加商品

分类名称:

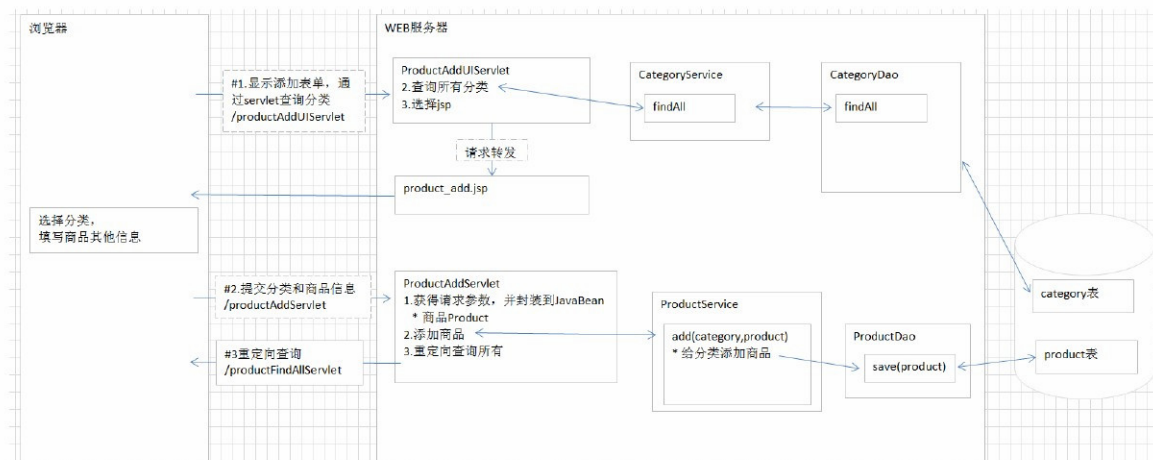
商品名称:

商品价格:

商品图片: 未选择任何文件

商品描述:

2.2 案例分析



1. 页面执行“/productAddUIServlet”显示“product_add.jsp”JSP 页面，在执行 servlet 时，需要通过查询所有分类。
2. 在“product_add.jsp”页面中，可以选择已有分类。
3. 提交添加表单到“/productAddServlet”，我们将表单数据封装到两个 JavaBean（Product），然后执行添加商品（add(product)）
4. 添加成功后，重定向到查询

2.3 案例实现

2.3.1 显示添加表单，确定入口

- 入口

文件位置：/day20/WebContent/admin/frame/left.jsp

```
left.jsp
38 书籍管理');
39 !, '添加书籍', '${pageContext.request.contextPath}/productAddUIServlet', '', 'mainFrame';
40 !, '查询书籍', '${pageContext.request.contextPath}/admin/product/product_list.jsp', 'mainFrame';
```

2.3.2 显示添加表单，编写 servlet

- 编写 servlet 实现类

```
public class ProductAddUIServlet extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //1 获得请求参数，并封装（此操作没有）
        //2 查询所有分类
```

```
CategoryService categoryService = new CategoryService();
List<Category> allCategory = categoryService.findAll();
//3 选择视图, product_add.jsp
//3.1 将查询结果存放到 request 作用域
request.setAttribute("allCategory", allCategory);
//3.2 请求转发到 jsp 页面
request.getRequestDispatcher("/admin/product/product_add.jsp")
    .forward(request, response);
}

public void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    doGet(request, response);
}
}
```

- web.xml 文件中, servlet 配置

```
<servlet>
    <servlet-name>ProductAddUIServlet</servlet-name>
    <servlet-class>cn.itcast.web.servlet.ProductAddUIServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ProductAddUIServlet</servlet-name>
    <url-pattern>/productAddUIServlet</url-pattern>
</servlet-mapping>
```

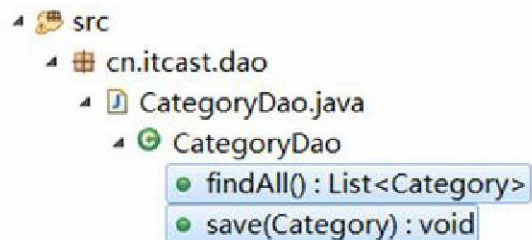
2.3.3 显示添加表单, 编写分类 service

```
public class CategoryService {

    private CategoryDao categoryDao = new CategoryDao();
    /**
     * 查询所有
     * @return
     */
    public List<Category> findAll(){
        return categoryDao.findAll();
    }

}
```

2.3.4 显示添加表单，编写分类 dao



- 分类 dao 实现：查询所有分类，保存分类

```
public class CategoryDao {  
  
    /**  
     * 查询所有  
     * @return  
     */  
    public List<Category> findAll(){  
        try {  
            QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());  
            String sql = "select * from category";  
            Object[] params = {};  
            return queryRunner.query(sql, new  
BeanListHandler<Category>(Category.class), params);  
        } catch (Exception e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

2.3.5 显示添加表单，展示分类信息

- 步骤 4: jsp 显示数据

位置: /day20/WebContent/admin/product/product_add.jsp

```
<select id="categorySelectId" name="cid" class="form-control" style="width: 30%">  
    <option value="">---选择已有分类---</option>  
    <c:forEach items="${allCategory}" var="category">  
        <option value="${category.cid}">${category.cname}</option>  
    </c:forEach>  
</select>
```

product_add.jsp

```
<select id="categorySelectId" name="cid" class="form-control" style="width: 30%">
  <option value="">--选择已有分类--</option>
  <c:forEach items="${allCategory}" var="category">
    <option value="${category.cid}">${category.cname}</option>
  </c:forEach>
</select>
<input type="text" id="categoryInputId" name="cname" placeholder="请输入新分类名称" title="请输入新分类名称" />
```

分类名称:

商品名称:

美食

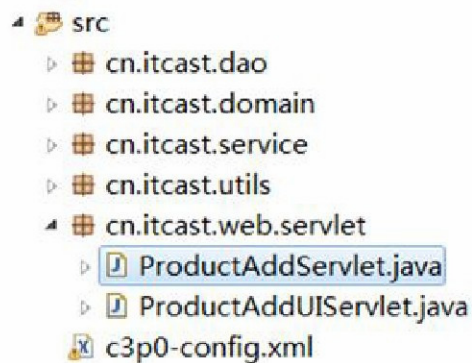
2.3.6 完善 JSP，确定表单内容

- 完善表单，摘要如下

```
<form action="${pageContext.request.contextPath}/AddProductServlet" method="post">
  商品名称: <input type="text" name="pname" />
  是否热门:
    <select name="is_hot">
      <option value="1">是</option>
      <option value="0">否</option>
    </select>
  市场价格: <input type="text" name="market_price" />
  商城价格: <input type="text" name="shop_price" />
  所属分类:
    <select name="cid">
      <c:forEach var="c" items="${ list }">
        <option value="${ c.cid }">${ c.cname }</option>
      </c:forEach>
    </select>
  商品描述: <textarea name="pdesc" rows="5" cols="30"></textarea>
```

2.3.7 添加功能，编写 servlet

- 编写 servlet 实现类，此时 ProductService 还没有编写，程序报错。



```
public class ProductAddServlet extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        //0 编码
        request.setCharacterEncoding("UTF-8");
        response.setContentType("text/html;charset=UTF-8");

        //1 封装数据到商品
        Product product = MyBeanUtils.populate(Product.class,
request.getParameterMap());

        //2 添加商品
        ProductService productService = new ProductService();
        productService.add(product);

        //3 重定向到查询页
        response.sendRedirect(request.getContextPath()
"/productFindAllServlet");

    }
}
```

- web.xml servlet 配置

```
<servlet>
    <servlet-name>ProductAddServlet</servlet-name>
    <servlet-class>cn.itcast.web.servlet.ProductAddServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ProductAddServlet</servlet-name>
    <url-pattern>/productAddServlet</url-pattern>
</servlet-mapping>
```

2.3.8 添加功能，编写 service

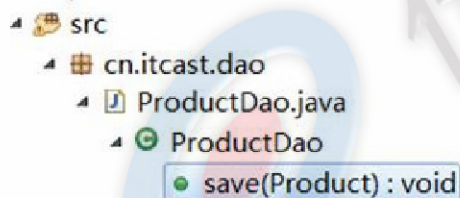
```
public class ProductService {

    private ProductDao productDao = new ProductDao();

    /**
     * 给指定的分类添加商品
     * @param product
     */
    public void add(Product product) {
        //2 给分类添加书籍
        productDao.save(product);
    }

}
```

2.3.9 添加功能，编写 dao 层



- 商品 dao 实现：保存商品

```
public class ProductDao {

    /**
     * 保存商品
     * @param category
     */
    public void save(Product product){
        try {
            QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
            String sql = "insert into product(pid,pname,price,
            pimage,description,category_id) values(?,?,?, ?,?,?)";
            Object[] params = {product.getPid() , product.getPname() ,
            product.getPrice(),
            product.getPimage() ,product.getDescription() ,product.getCategory_id()};
```

```
        queryRunner.update(sql, params);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

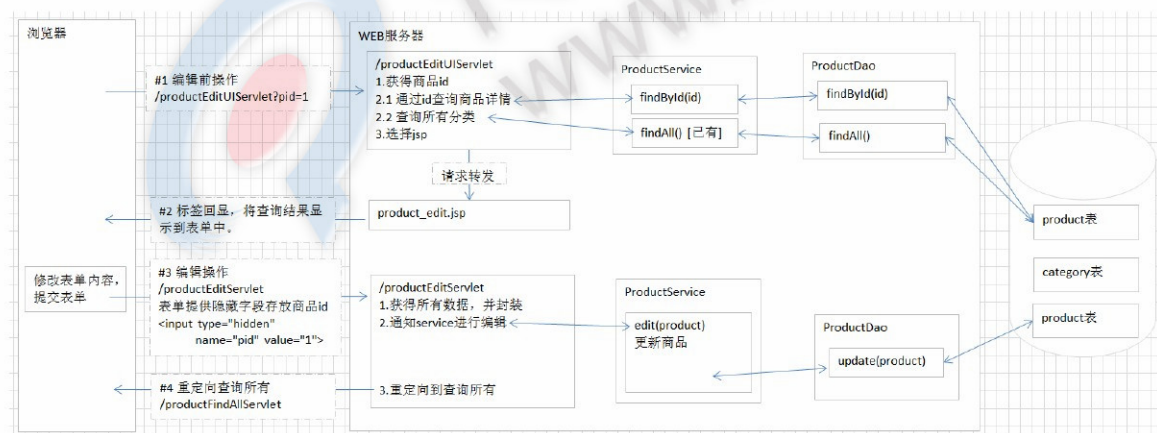
第3章 案例：修改商品

3.1 案例介绍

在日常的上网中，我们都修改的“用户信息”，首先可以看到上次填写的信息，然后将不符合的内容修改掉，最后保存修改。整个过程就是“修改”的操作流程。

1. 通过标识 ID 查询详细信息
2. 在修改表单中回显具体信息
3. 用户修改表单信息，并提交表单
4. 将表单数据，为 ID 为标识更新到数据库。

3.2 案例分析



从查询所有列表点击编辑，浏览器发送请求“/productEditUIServlet?pid=1”并将需要查询的商品的 id 传递给服务器，servlet 获得商品 id，通过 id 查询商品详情，然后将查询结果存放到 request 作用域，使用请求转发，将数据显示在“product_edit.jsp”的表单中，并提供隐藏字存放商品 id。当用户修改好数据后，提交表单到“/productEditServlet”进行编辑处理。

3.3 案例实现

3.3.1 修改 jsp，确定入口

- 位置：/day14_edit/WebContent/admin/product/product_list.jsp

```
<%--编辑 --%>

<a href="${pageContext.request.contextPath}/productEditUIServlet?pid=${product.pid}">
    
</a>
```

product_list.jsp

```
138 <%--编辑 --%>
139 <a href="${pageContext.request.contextPath}/productEditUIServlet?pid=${product.pid}">
140     
141 </a>
```

3.3.2 编辑前操作

- 步骤 1：编写编辑前 servlet 实现类（ProductEditUIServlet）

- └─ cn.itcast.web.servlet
 - └─ ProductAddServlet.java
 - └─ ProductAddUIServlet.java
 - └─ ProductEditUIServlet.java
 - └─ ProductFindAllServlet.java

```
public class ProductEditUIServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //1 获得商品 id
        String pid = request.getParameter("pid");

        //2.1 通过 id 查询商品详情
        ProductService productService = new ProductService();
        Product product = productService.findById(pid);

        //2.1 通过所有的分类
        CategoryService categoryService = new CategoryService();
        List<Category> allCategory = categoryService.findAll();

        //3.1 将查询数据存放在 request 作用域
        request.setAttribute("product", product);
        request.setAttribute("allCategory", allCategory);
    }
}
```

```
//3.2 请求转发到jsp 页面
```

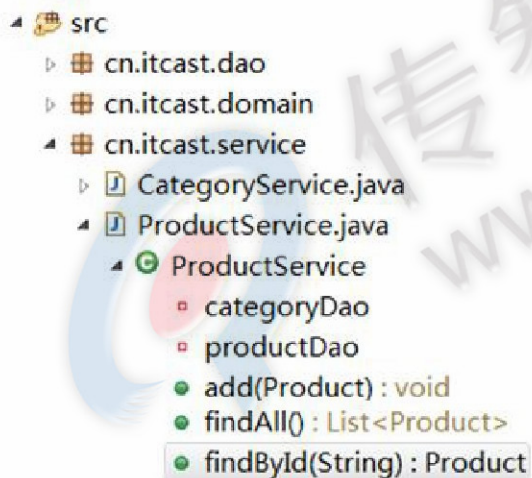
```
request.getRequestDispatcher("/admin/product/product_edit.jsp").forward(request,
response);

}
```

- web.xml 配置编辑前 servlet

```
<servlet>
    <servlet-name>ProductEditUIServlet</servlet-name>
    <servlet-class>cn.itcast.web.servlet.ProductEditUIServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ProductEditUIServlet</servlet-name>
    <url-pattern>/productEditUIServlet</url-pattern>
</servlet-mapping>
```

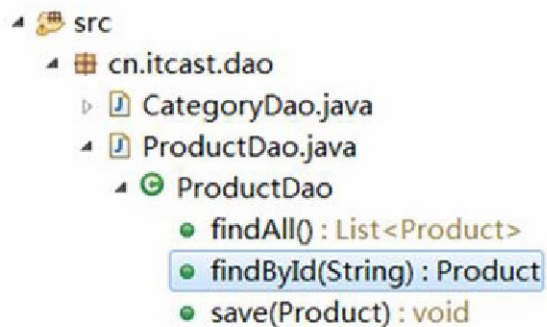
- 步骤 2: 编写 ProductService 类, 提供方法 findById(id)



The image shows the Eclipse IDE project structure. The 'src' folder is expanded, showing the package hierarchy: cn.itcast.dao, cn.itcast.domain, and cn.itcast.service. Inside cn.itcast.service, there are two Java files: CategoryService.java and ProductService.java. The ProductService.java file is selected, and its class structure is visible, showing two private attributes: categoryDao and productDao, and three public methods: add(Product) : void, findAll() : List<Product>, and findById(String) : Product. The findById method is highlighted with a red box.

```
/**
 * 通过 id 查询详情
 * @return
 */
public Product findById(String pid){
    return productDao.findById(pid);
}
```

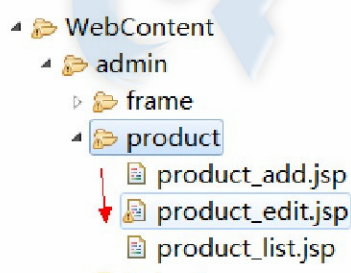
- 步骤 3: 编写 ProductDao 类, 提供两个方法 findById(id)



通过 id 查询商品

```
/**
 * 通过 id 查询详情，含分类
 * * 编辑时，所有分类列表中，当前分类需要选中
 * @return
 */
public Product findById(String pid){
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        String sql = "select * from product p , category c where p.category_id = c.cid
and p.pid = ?";
        Object[] params = {pid};
        return queryRunner.query(sql, new BeanHandler<Product>(Product.class),
params);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

- 步骤 4: 复制 “product_add.jsp”，重命名为 “product_edit.jsp”，并进行表单回显



表单回显概要如下：

```
<form action="${pageContext.request.contextPath}/productEditServlet" method="post" >
<!--提供隐藏字段 -->
<input type="hidden" name="pid" value="${product.pid}"/>

<select id="categorySelectId" name="cid" class="form-control" style="width: 30%">
    <option value="">---选择已有分类---</option>
    <c:forEach items="${allCategory}" var="category">
        <!-- 将当前商品对应的分类选中 -->
```

```

        <option ${product.category.cname == category.cname ? "selected='selected'" :
""} value="${category.cid}">${category.cname}</option>
    </c:forEach>
</select>

```

商品名称: <input type="text" name="pname" value="\${product.pname}" .../>

商品价格: <input type="text" name="price" value="\${product.price}" .../>

商品描述: <textarea name="description" ...>\${product.description}</textarea>

product_edit.jsp ProductDao.java ProductEditServlet.java

```

60<tr>
61<td class="ta_01" align="center" bgColor="#afd1f3" colSpan="4" hei
62<strong>添加商品</strong>
63<!--提供隐藏字段 --%>
64<input type="hidden" name="pid" value="${product.pid}"/>
65</td>
66<tr>

```

3.3.3 编辑操作

- 步骤 1: 编写编辑 servlet 实现类 (ProductEditServlet)

```

public class ProductEditServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // 0 编码
        request.setCharacterEncoding("UTF-8");

        // 1 获得数据, 并封装
        Product product =
            MyBeanUtils.populate(Product.class, request.getParameterMap());

        // 2 编辑操作
        ProductService productService = new ProductService();
        productService.edit(product);

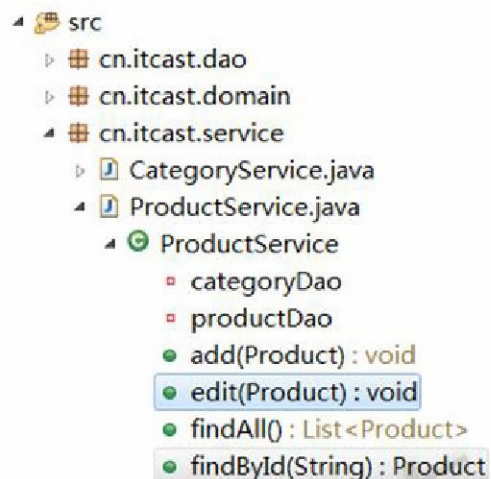
        // 3 重定向查询所有
        response.sendRedirect(request.getContextPath()
            + "/productFindAllServlet");
    }
}

```

- web.xml 配置编辑 servlet

```
<servlet>
    <servlet-name>ProductEditServlet</servlet-name>
    <servlet-class>cn.itcast.web.servlet.ProductEditServlet</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ProductEditServlet</servlet-name>
    <url-pattern>/productEditServlet</url-pattern>
</servlet-mapping>
```

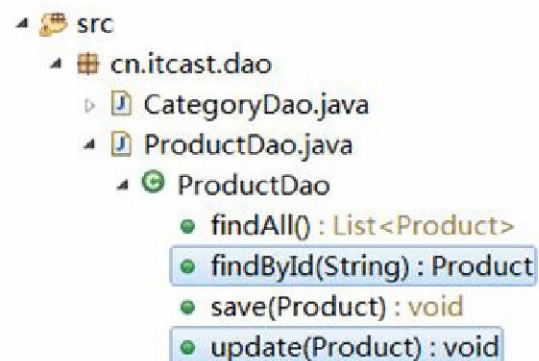
- 步骤 2: 编写 ProductService 类, 提供方法 edit(product)



```
src
├── cn.itcast.dao
├── cn.itcast.domain
└── cn.itcast.service
    ├── CategoryService.java
    └── ProductService.java
        ├── ProductService
        │   ├── categoryDao
        │   ├── productDao
        │   ├── add(Product) : void
        │   └── edit(Product) : void
        │   ├── findAll() : List<Product>
        │   └── findById(String) : Product
```

```
/**
 * 编辑商品
 * @param category
 */
public void edit(Product product){
    productDao.update(product);
}
```

- 步骤 3: 编写 ProductDao 类, 提供方法 update(product)



```
src
├── cn.itcast.dao
│   ├── CategoryDao.java
│   └── ProductDao.java
│       ├── ProductDao
│       │   ├── findAll() : List<Product>
│       │   ├── findById(String) : Product
│       │   ├── save(Product) : void
│       │   └── update(Product) : void
```

```
/**
 * 更新商品
 * @param category
 */
```

```

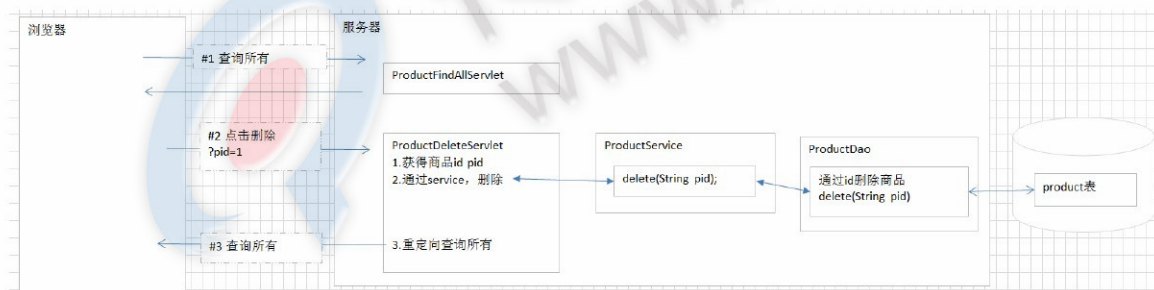
public void update(Product product){
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        String sql = "update product set pname=?,price=?,
        pimage=?,description=?,category_id=? where pid=?";
        Object[] params = {product.getPname() , product.getPrice(),

        product.getPimage() ,product.getDescription() ,product.getCategory().getCid(),
        product.getPid()};
        queryRunner.update(sql, params);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
}

```

第4章 案例：删除指定商品

4.1 案例分析



4.2 案例实现

- 步骤 1: 修改 product_list.jsp, 修改删除确定删除 servlet 和要删除商品 id

```

product_list.jsp
147<td align="center" style="HEIGHT: 22px">
148<a href="${pageContext.request.contextPath}/ProductDeleteServlet?pid=${product.pid}">
149
151</td>

```

```

<a href="${pageContext.request.contextPath}/ProductDeleteServlet?pid=${product.pid}">

```

- 步骤 2: 编写 ProductDeleteServlet

```

public class ProductDeleteServlet extends HttpServlet {

```

```
public void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    //1 获得所有的 id
    String pid = request.getParameter("pid");
    //2 删除
    ProductService productService = new ProductService();
    productService.delete(pid);

    //3 重定向查询所有
    response.sendRedirect(request.getContextPath()
        + "/productFindAllServlet");
}
```

● 步骤 3: 编写 service

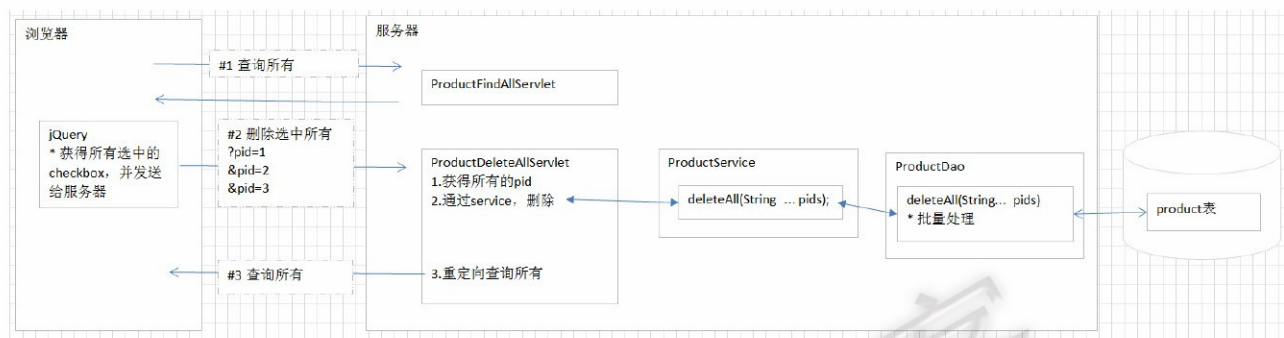
```
/**
 * 删除所有
 * @param pids
 */
public void delete(String pid) {
    productDao.delete(pid);
}
```

● 步骤 4: 编写 dao

```
/**
 * 通过 id 删除
 * @param pid
 */
public void delete(String pid) {
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
        //删除 sql 语句
        String sql = "delete from product where pid = ?";
        queryRunner.update(sql, pid);
    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
```

第5章 案例：删除选中商品

5.1 案例分析



5.2 案例实现

- 步骤 1: 修改 jsp, 完善 checkbox, name 为 pid, value 为商品 id

```
product_list.jsp
115 <!--列表项 start -->
116 <c:forEach items="${allProduct}" var="product">
117   <tr onmouseover="this.style.backgroundColor = '#F5FAFE'"
118     onmouseout="this.style.backgroundColor = '#fff';">
119     <td style="CURSOR: hand; HEIGHT: 22px" align="center">
120       <input type="checkbox" name="pid" value="${product.pid}" />
121     </td>
122   </c:forEach>
```

- 步骤 2: 编写 js 代码, 获得所有的 pid

添加 id 属性: `<a id="deleteAllId" href="javascript:void(0)">删除选中</a>`

```
<script type="text/javascript">
    $(function(){
        //绑定点击事件
        $("#deleteAllId").click(function(){
            //所有选中的 checkbox
            var all = $("input[name='pid']:checked");
            if(all.length == 0){
                alert("请选择需要上传的商品");
                return false;
            }
            //
            if(window.confirm("您确定要删除所选"+all.length+"个商品吗? ")){
                //获得所有的 id 串
```

```
        var ids = all.serialize();

        location.href="${pageContext.request.contextPath}/productDeleteServlet?" + ids;
    } else {
        $("input[name='pid']").removeProp("checked");
    }
    });
});
</script>
```

● 步骤 3: 编写 servlet

```
public class ProductDeleteAllServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        //1 获得所有的 id
        String[] pids = request.getParameterValues("pid");
        //2 删除
        ProductService productService = new ProductService();
        productService.deleteAll(pids);

        //3 重定向查询所有
        response.sendRedirect(request.getContextPath() +
"/productFindAllServlet");
    }
}
```

● 步骤 4: 编写 service

```
/**
 * 删除所有
 * @param pids
 */
public void deleteAll(String... pids) {
    productDao.delete(pids);
}
```

● 步骤 5: 编写 dao

```
/**
 * 删除所有
 * @param pids
 */
public void deleteAll(String... pids) {
    try {
        QueryRunner queryRunner = new QueryRunner(C3P0Utils.getDataSource());
```

```
//删除 sql 语句
String sql = "delete from product where pid = ?";
//将所有的 pid 拼凑一个二维数组，列表示 id，行表示个数
Object[][] params = new Object[pids.length][];
for (int i = 0; i < pids.length; i++) {
    params[i] = new Object[]{pids[i]};
}
queryRunner.batch(sql, params);
} catch (Exception e) {
    throw new RuntimeException(e);
}
}
```

第6章 总结



传智播客
www.itcast.cn